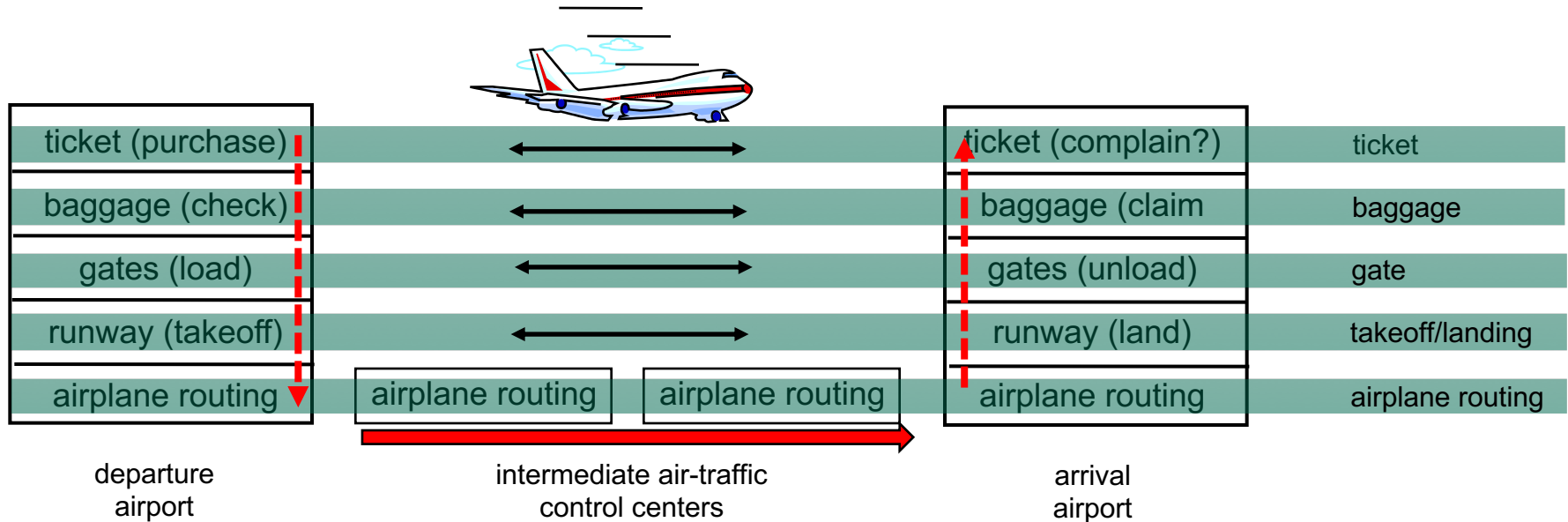


Layering of airline functionality



layers: each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

Why layering?

- *layered* structure allows breakdown of a complex system into smaller, well identified *modules* with explicit responsibilities/relationships to each other
 - layered reference model – *up/down relationships only - well defined*
- layering considered harmful?
 - Overlap in functionality (e.g., error control), dependency wrt specific requirements (e.g., timestamps) – not fully independent

Why modules?

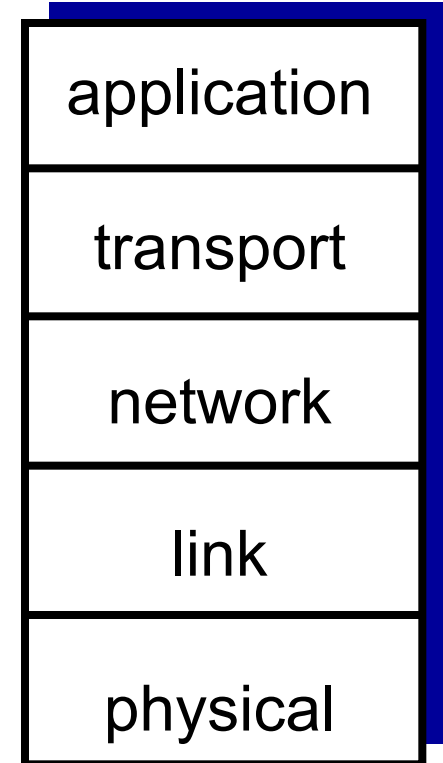
- *modularization* eases maintenance/updating of system
 - change of implementation of a layer's service transparent to rest of system – internals are hidden
 - e.g., change in gate procedure doesn't affect rest of system *operation*, a flight is still a flight, passenger still gets to destination
 - but a change in a layer's service could impact *performance* of end system -> passengers may not be happy with a new gate procedure – unless reflected in their ticket purchase

Is layering the right choice?

- is layering harmful?
 - overlap in functionality (e.g., error control),
 - dependency with respect to specific requirements (e.g., timestamps) – not fully independent

Internet protocol stack - layers

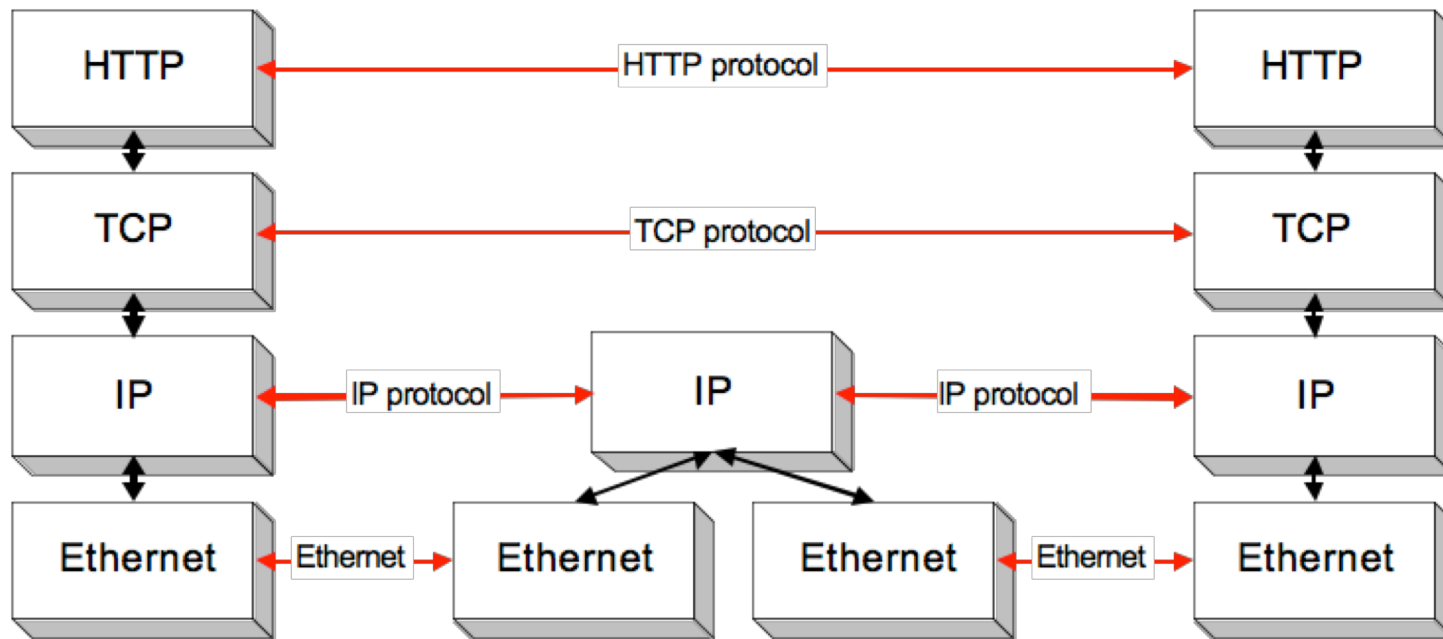
- **application:** supporting networked applications, process to process transfer – data stream → managed as *messages*
 - FTP, SMTP, HTTP,....
- **transport:** process to process message transfer → managed as *segments/datagrams*
 - TCP, UDP
- **network:** routing of data segments from source to destination hop by hop → managed as *datagrams*
 - IP (Internet Protocol)
- **link:** data transfer between neighboring network elements → managed as *frames*
 - Ethernet, 802.11 (WiFi), PPP
- **physical:** data “on the wire” → managed as *bits*
 - optical, electrical, electromagnetic.....



Layers: Functionality & Services

- **Data Link Layer:** Transmission of frames
 - **Functions:** Framing, media access control, error checking, flow control
- **Network Layer:** Forwarding of packets (datagrams)
 - **Functions:** Network addressing, hop by hop routing, header checking, loop prevention
- **Transport Layer:** Transfer of “chunks of application data”
 - **Functions:** Connection establishment/termination, error control, flow control, congestion control
- **Application Layer:** Application specific – display/presentation
 - **Functions:** Synchronization/timing, error recovery

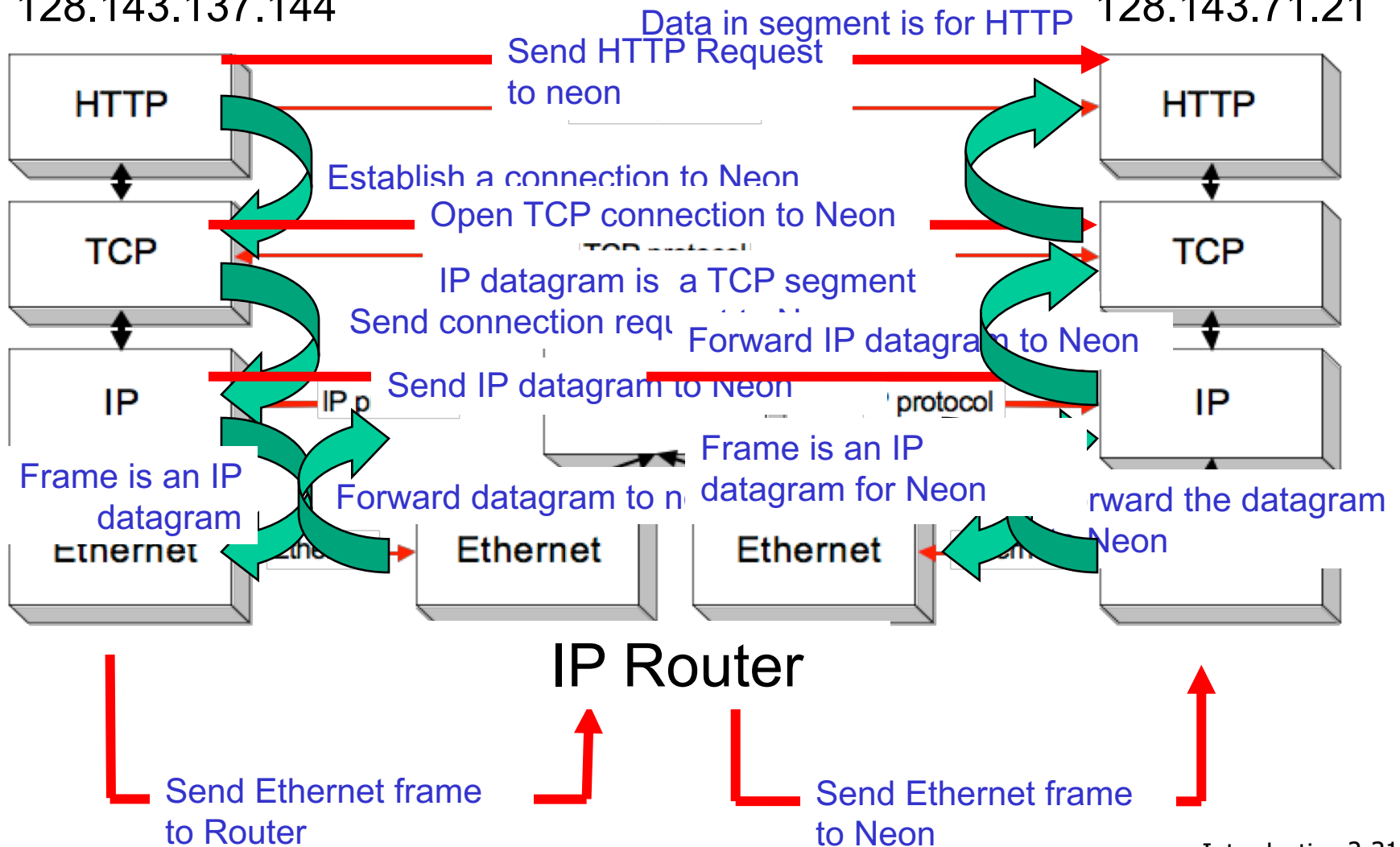
IEFT Protocol Stack Example



Protocols in Action

Argon
128.143.137.144

Neon
128.143.71.21



Encapsulation

- with layering comes encapsulation
- each layer adds control bits used to process the information to provide the desired type of service to the layer above it and ultimately to the application
- the packet grows in size as it drops from application to datalink at the source
- the packet is stripped of bits as it travels back up the stack to the application layer

IETF Encapsulation

link layer frame with header H_l



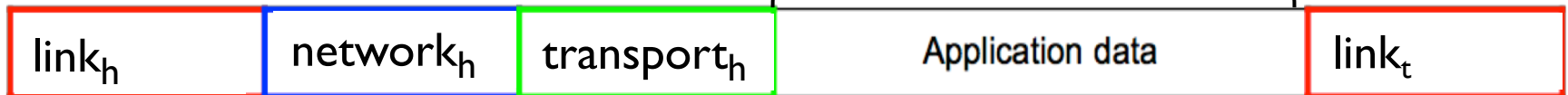
network layer datagram with header H_n



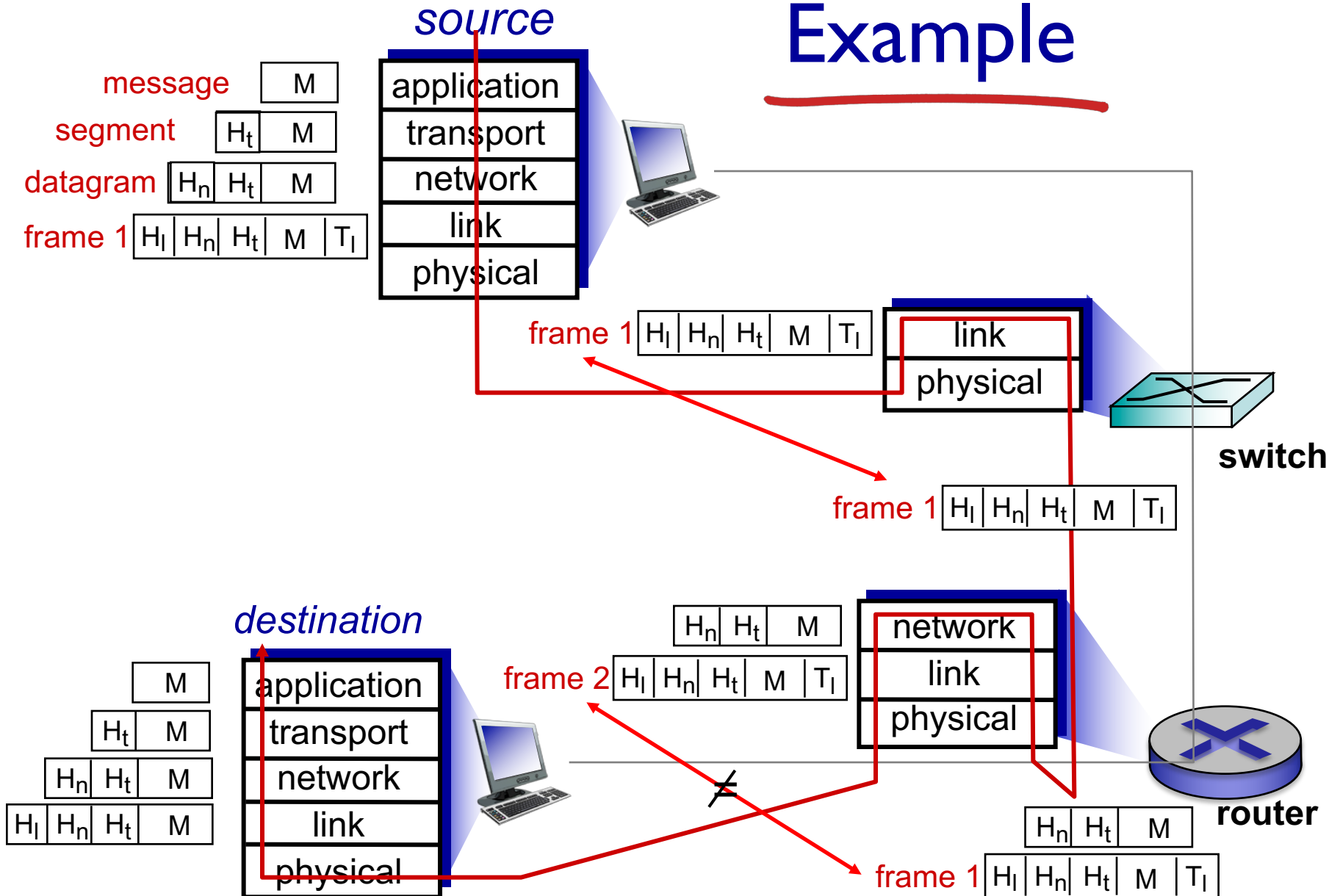
transport layer segment with header H_t



message



Example



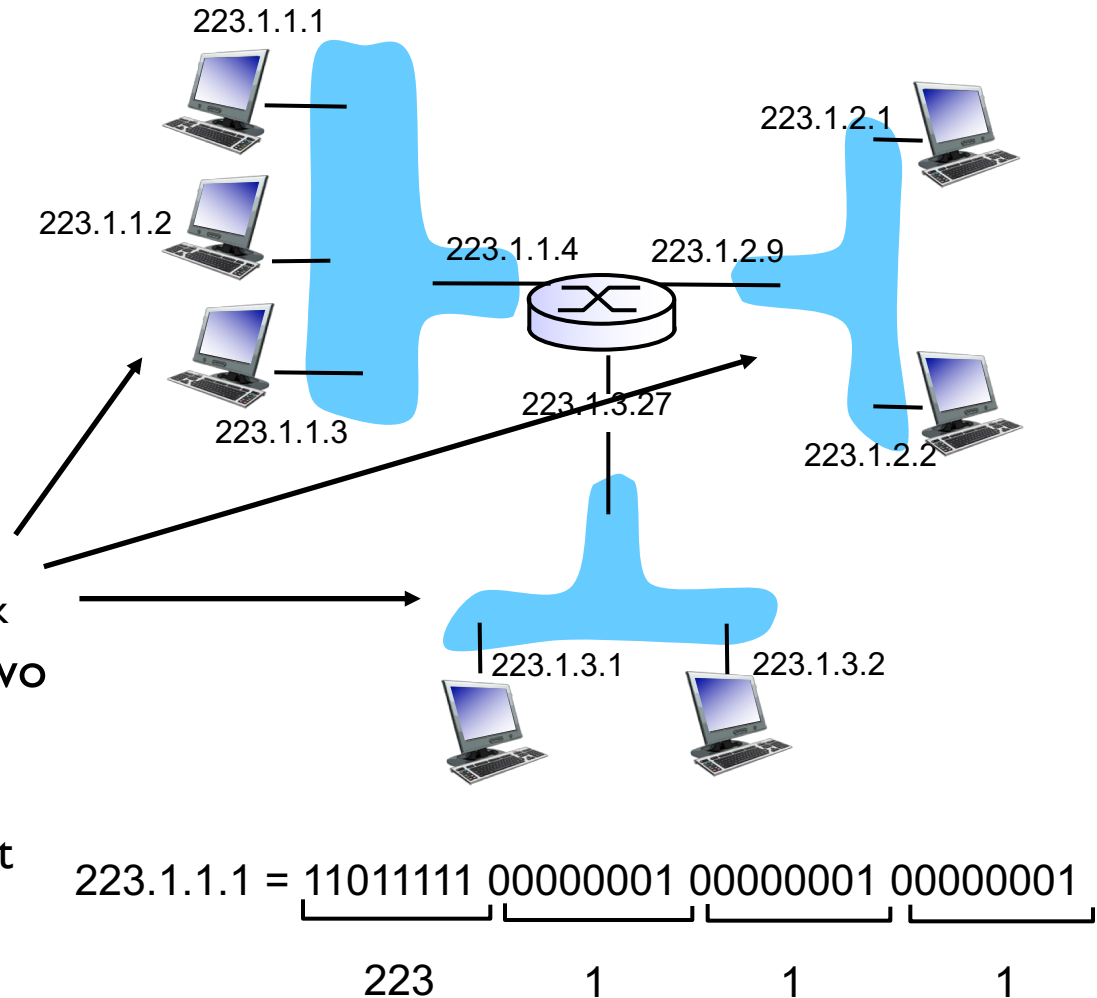
Chapter 4: Network Layer

4.3 IP: Internet Protocol

- IPv4 addressing

Internet Addressing: Introduction

- **Internet address:** 32-bit identifier for host, router interface
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - each interface **creates** a distinct/separate network
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)
 - interfaces may or may not be on a same network
- **Internet address associated with each interface**

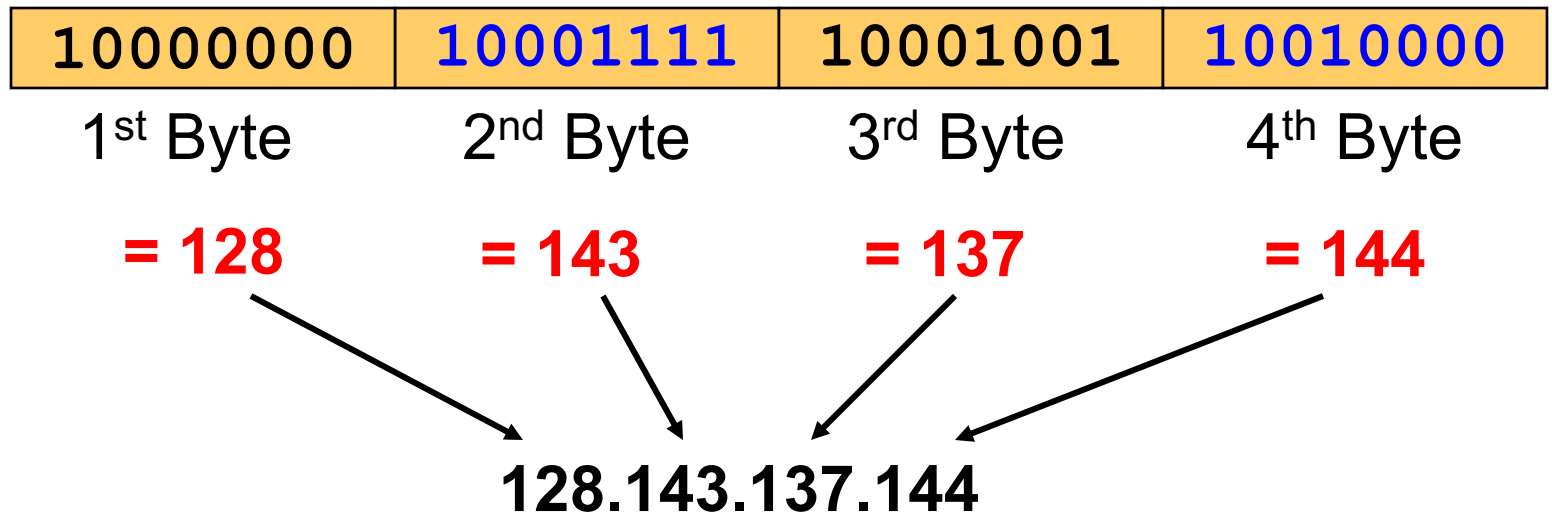


What is an Internet address?

- Internet address is usually referred to as the IP (Internet Protocol) address.
- henceforth we will refer to it as IP address
- an IP address is a unique global address for a network interface
- an IP address:
 - is a **32 bit long** identifier – 4bytes
 - encodes a network number referred to as **network prefix** and a device number referred to as **host address**
- human analogy:
 - network prefix → street name
 - host number → house number

IP Dotted Decimal Notation

- IP addresses are written in a **dotted decimal format**
- each byte is identified by a decimal number in the range $[0 \dots 255] - 2^8$
- example



Network prefix and host number

- the network prefix identifies a network and the device number identifies a specific device (actually, *interface* on the network as a device can have more than 1 "network card", and each card will have a unique IP address).
- real life analogy? - street name and house number

network prefix

host number

- **how do we know how long the network prefix is?**
 - the network prefix **used** to be implicitly defined (**class-based addressing, A,B,C,D...**)
 - the network prefix now is **flexible** and is indicated by a **prefix/netmask (classless)**.

Example

Example: argon.cs.virginia.edu

- IP address is 128.143.137.144
 - is that enough info to route/forward a datagram??? -> **No**.
 - must indicate a network prefix for every IP device (host and router)
- using prefix notation an IP address is: **128.143.137.144/x**
 - e.g., **x = 16** means network prefix is 16 bits long
- the prefix is identified by a network mask: prefix = 16 → mask consists of 16 'one's'
- i.e., mask is represented as: **255.255.0.0** or hex format: **ffff0000**
 - > **network prefix (or ID or address)** (IP address **AND** netmask) is:
128.143.0.0
 - > **host number** (IP address **AND** inverse of netmask(=0000ffff) is:
0.0.137.144

